

CSAI810 Project Definition

Group Category: CSAI810 Groups

Group Name: CSAI 810 Section 44

1. Title of your project

Autonomous Code Agent

2. Short description of goals and limitations

Goals:

The main goal is to build a comprehensive, local-first AI coding assistant that can autonomously help developers build software. Unlike standard chatbots, the Autonomous Code Agent enables the LLM to actively interact with the development environment—writing files, executing terminal commands, managing git repositories, and performing web searches to solve complex coding tasks. It aims to bridge the gap between "chatting about code" and "doing the coding."

Limitations:

- **Context Window:** The agent's understanding of large codebases is limited by the context window of the underlying models (though retrieval techniques are used).
- **Visual Validation:** The agent cannot "see" the UI it builds (no computer vision integration yet), so it relies on text-based feedback and user verification.
- **Model Dependency:** Performance heavily depends on the reasoning capability of the selected model (e.g., GPT-4o vs. Llama 3).

3. What will you achieve (learn + produce) and what is your prior knowledge of this task?

Produce:

We will produce a fully functional Streamlit-based application (deployable as a Local Web App or Desktop Interface) that serves as the interface for the agent. The system will include a persistent memory layer (SQLite), a robust toolset (File I/O, Git, Terminal, Search), and a flexible model switcher supporting both local (Ollama) and cloud (OpenRouter, Google Gemini) models.

Learn:

We will learn how to:

- Design tool-use loops (ReAct pattern) for LLMs.
- Manage structured output and function calling across different model providers.
- Implement persistent conversation history and project context management.
- Integrate local development tools (Git, Terminal) safely with an AI agent.

Prior Knowledge:

We have prior experience with Python programming, API integrations (REST), and basic prompting. We are familiar with the concept of Agents but want to deepen our understanding of building a cohesive "Agentic" system.

4. Why do you want to do this project?

We want to move beyond simple "Chat Q&A" interfaces. Real developer productivity comes from an agent that can *act*—scaffold projects, fix bugs directly in files, and handle version control. This project allows us to explore the cutting edge of Agentic AI while building a tool that we can actually use in our daily workflow.

5. What will be the final deliverable and by whom could it be used?

Final Deliverable:

A software package (Python/Streamlit) containing:

- The "Code Agent" application.
- A suite of integrated tools (File System, Terminal, Git, Search).
- A unified UI for chat, workspace management, and model selection.
- Documentation for setup and usage.

Target User:

Software developers, students, and hobbyists who want an AI assistant that runs locally and integrates directly with their project files, rather than copying-pasting code from a web browser.

6. Project boundaries (30 Hours Scope)

Included (In-Scope):

- **Core Agent Loop:** Implementation of the thinking/acting loop using LiteLLM/Google Generative AI.

Tool Implementation:

- File System (Read/Write/List)
- Terminal (Run safe commands)
- Git (Commit, Push, Pull)
- Web Search (Tavily/DuckDuckGo)
- **UI:** A polished Streamlit interface with a sticky header, chat history, and sidebar for workspace management.
- **Project Management:** Ability to switch between different "project" folders/contexts.
- **Memory:** Storing chat history in a local SQLite database.

Excluded (Out-of-Scope):

- **Multi-User Auth:** The app is designed for single-user local usage.
- **Visual UI Testing:** Writing automated Selenium tests or visual regression testing.

- **IDE Extensions:** We are building a standalone app, not a VS Code extension (though it acts similarly).
- **Voice Mode:** Text-only interaction.